Motivos de insatisfação a partir do CustomerFeedback — documentação do código

Nov 10, 2026 · @Thiago

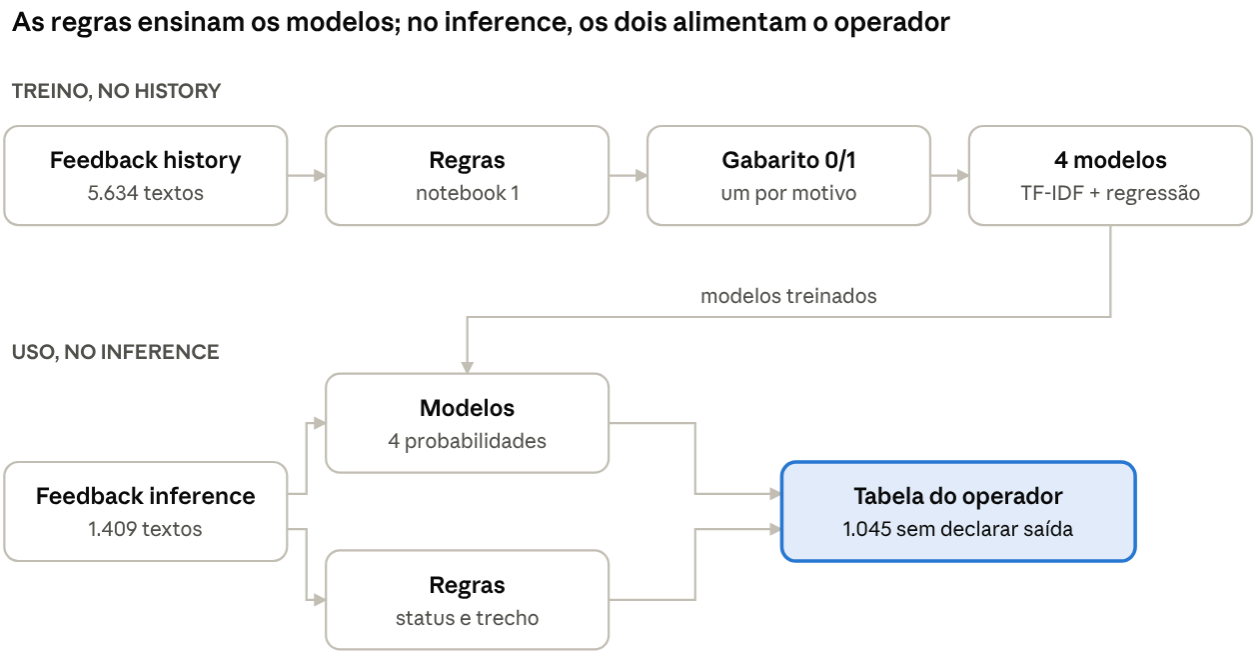
1. Visão geral

Dois notebooks transformam o texto do `CustomerFeedback` em sinais para o operador. O primeiro **lê** o texto com regras. O segundo usa essas regras como gabarito e treina um modelo que devolve **uma probabilidade por motivo de insatisfação**.

Por que isso não entra no modelo de churn. No `history`, 1.498 dos 1.504 clientes que saíram escrevem no feedback que decidiram sair. Usar o texto como feature seria dar a resposta ao modelo (vazamento). Por isso o texto vira uma **camada de apoio**, separada do modelo de escoragem.

Para quem é. O foco são os clientes que **não** declararam saída: 1.045 dos 1.409 do `inference`. Para eles, a pergunta é: o feedback já mostra alguma queixa que o operador pode tratar antes que ela vire cancelamento?

Notebook	Entrada	O que faz	Saída
`motivos_cancelamento`	Texto do feedback	Regras de palavras (expressões regulares)	`status_texto`, um 0/1 por motivo, trecho de evidência
`prob_motivos_feedback`	Texto + o 0/1 das regras (history)	TF-IDF + regressão logística, um modelo por motivo	`prob_preco`, `prob_concorrente`, `prob_pagamento`, `prob_tecnico`



fluxo dos dois notebooks · treino no history, uso no inference

No `history`, as regras criam o gabarito que treina os modelos. No `inference`, o mesmo texto passa pelas regras e pelos modelos, e os dois resultados chegam juntos ao operador.

Os quatro motivos foram escolhidos porque cada um leva a uma ação diferente do operador: **preço**, **oferta do concorrente**, **forma de pagamento** e **técnico** (velocidade e instabilidade).

2. Notebook 1, fase 1: o cliente declarou que vai sair?

A função `status_texto` coloca cada feedback em um de três grupos. No `history` ela acerta 100% de quem marca como "declarou saída" e encontra 99,6% de quem saiu.

A ferramenta: expressão regular (regex). É um padrão de busca de texto. `decided to (?:churn|cancel)` encontra "decided to churn" e "decided to cancel". O `|` significa "ou", e o `re.I` faz a busca ignorar maiúsculas e minúsculas.

O código define três listas de padrões:

Padrão	Procura	Exemplos que encontra
`DECLAROU`	Frases em que o cliente afirma a decisão de sair	"I have decided to churn", "had to cancel", "ended up switching", "I recently cancelled"

Padrão	Procura	Exemplos que encontra
`NEGA`	Frases que negam a saída, ou falam de ter vindo **para** esta empresa	"no plans to switch", "would not leave", "happy with my decision to switch to this provider"
`INSATISFEITO`	Insatisfação forte sem decisão declarada	"disappointed", "dissatisfied", "considering switching"

A função testa nesta ordem:

1. Encontrou `DECLAROU` e **não** encontrou `NEGA` → `1_declarou_saida`.
2. Senão, encontrou `INSATISFEITO` → `2_insatisfeito`.
3. Senão → `3_satisfeito`.

Por que existe o `NEGA`. Sem ele, o único erro da primeira versão era um cliente satisfeito que escreveu "I am happy with my **decision to switch** to this provider". Ele trocou **para** a empresa, não **dela**. O `NEGA` corrige esse tipo de frase.

Os números prefixados (1_, 2_, 3_) servem só para a coluna ordenar do mais urgente para o menos urgente.

3. Notebook 1, fase 2: quais motivos aparecem em tom de reclamação?

Um motivo só é marcado quando o **mesmo trecho** do texto fala do assunto **e** reclama dele. Citar o assunto não basta: "the monthly charges were reasonable" fala de preço, mas não é queixa.

As peças do código

Peça	Para que serve	Exemplo
`MOTIVOS`	Um padrão de palavras por assunto	preço: `charge`, `price`, `expensive`, `$` seguido de número
`QUEIXA`	Palavras que indicam reclamação	`too`, `high`, `slow`, `inconvenient`, `outdated`, `issue`, `frequent`
`NEGA_QUEIXA`	Frases que parecem queixa, mas negam o problema	"never had any issues", "without any interruptions", "hassle-free"

Peça	Para que serve	Exemplo
`SEM_EXIGIR_QUEIXA`	Motivos que contam mesmo sem palavra de queixa	concorrente, pessoal, sem internet, aumento de preço (o fato já é o motivo)
`CORTE` e `trechos()`	Quebram o texto em pedaços	corta em `.`, `!`, `?`, `;` e antes de "but", "however", "while", "although"
`extrair_motivos()`	Aplica as regras trecho a trecho	devolve a lista de motivos e a frase de evidência de cada um
`motivo_principal()`	Escolhe um motivo para destaque	o citado na frase em que o cliente declara a saída; senão, o primeiro do texto
`processar()`	Junta tudo numa tabela	uma linha por cliente

Exemplo passo a passo

Texto real: *"While the fiber optic internet was fast and reliable, I found the monthly charges to be on the higher side at $69.0."*

1. **Cortar em trechos.** O corte antes de "while" e na vírgula gera dois pedaços: A = *"the fiber optic internet was fast and reliable"* e B = *"I found the monthly charges to be on the higher side at $69.0"*.
2. **Perguntar o assunto.** A fala de instabilidade (`reliable`). B fala de preço (`charges` , `$69`).
3. **Perguntar se é queixa.** A não tem palavra de queixa. B tem `higher` .
4. **Resultado.** Só B vira motivo: **preço**. A evidência guardada é o trecho B.

Sem o corte do passo 1, a frase inteira teria `reliable` e `higher` juntas, e a regra marcaria "instabilidade" por engano. Foi o erro da primeira versão.

Por que cada ajuste foi feito

Cada ajuste corrigiu um erro visto nos dados. O teste foi sempre o mesmo: clientes satisfeitos não deveriam aparecer com muitos motivos.

Ajuste	Erro que corrigiu
Cortar em trechos, não em frases	"fast and reliable, but too expensive" virava instabilidade

Ajuste	Erro que corrigiu
`NEGA_QUEIXA`	"I never had any issues" virava instabilidade
`\bhigh\b` em vez de `high`	"**high**ly recommend" virava queixa
Concorrente só com oferta melhor	"switch to a different provider" é o destino, não o motivo

Antes desses ajustes, 66% dos satisfeitos tinham "motivo preço". Depois, 4%.

Colunas que saem desta fase

- `motivo_preco`, `motivo_concorrente`, `motivo_pagamento`, `motivo_instabilidade`, `motivo_velocidade`, `motivo_sem_internet` e outros: 1 se o motivo apareceu, 0 se não.
- `motivos`: a lista, na ordem em que aparecem no texto.
- `motivo_principal`: um só, para destaque.
- `trecho_evidencia`: a frase do cliente que sustenta o motivo principal.

4. Notebook 1, fases 3 e 4: validação no history e aplicação no inference

No `history` sabemos quem saiu, então dá para conferir as regras. No `inference` não sabemos, então só aplicamos.

Fase 3: o status confere com o Churn real?

O `pd.crosstab` cruza o `status_texto` com o `Churn`:

status_texto	Ficou	Saiu
1_declarou_saida	0	1.498
2_insatisfeito	3	6
3_satisfeito	4.127	0

- **Precisão = 100%.** Dos 1.498 que a regra marcou como "declarou saída", todos saíram.
- **Cobertura = 99,6%.** Dos 1.504 que saíram, a regra encontrou 1.498.

Isso confirma o vazamento: o texto não *prevê* o churn, ele *conta* o churn.

Fase 3: os motivos fazem sentido?

Não existe um "motivo verdadeiro" para comparar. O teste usado foi indireto: quem ficou deveria ter poucas queixas.

Motivo	% entre quem saiu	% entre quem ficou
Instabilidade	47%	1%
Preço	42%	4%
Oferta do concorrente	38%	0%
Forma de pagamento	31%	2%
Velocidade	29%	1%
Pessoal (mudança, financeiro)	8%	0%

Os 2% de pagamento entre quem ficou são queixas reais, como "the only downside is that I have to pay by mailed check".

Fase 4: o resultado no inference

status_texto	Clientes
1_declarou_saida	364
2_insatisfeito	1
3_satisfeito	1.044

Os 364 (26%) já dizem no texto que vão sair, proporção parecida com a do `history` (26,7%). Os outros 1.045 são o público do notebook 2.

5. Notebook 2, fases 1 e 2: o gabarito e o modelo

As regras dizem 0 ou 1. O modelo aprende com esse 0/1 e devolve uma **probabilidade de 0 a 1**, o que permite ordenar os clientes pela força da queixa e reconhecer frases que as regras não previam.

Fase 1: o gabarito (`GABARITO`)

A função `processar` do notebook 1 roda em todo o `history`. As colunas de motivo são agrupadas em quatro, com o operador `|` ("ou"): basta uma das colunas ser 1.

Motivo do modelo	Junta as colunas	Textos com o motivo
preço	`motivo_preco` ou `motivo_aumento_preco`	807 (14,3%)
concorrente	`motivo_concorrente`	574 (10,2%)
pagamento	`motivo_pagamento`	541 (9,6%)
técnico	`motivo_instabilidade` ou `motivo_velocidade` ou `motivo_sem_internet`	872 (15,5%)

O gabarito vale para **todos** os clientes. Um satisfeito que reclama do cheque também recebe 1 em pagamento. O `Churn` não entra aqui.

Fase 2: o modelo (`novo_modelo`)

Cada motivo ganha seu próprio modelo, porque um texto pode ter vários motivos ao mesmo tempo. O modelo é um `Pipeline` de duas etapas: a primeira transforma o texto em números, a segunda aprende.

Etapa 1: `TfidfVectorizer`. Cria uma coluna por termo, com o peso TF-IDF daquele termo no texto.

Parâmetro	Valor	O que faz
`ngram_range`	(1, 3)	Usa palavras soltas, pares e trios: "check", "check inconvenient", "lack internet service"
`min_df`	5	Descarta termos que aparecem em menos de 5 textos (ruído)
`stop_words`	`STOP`	Remove palavras vazias do inglês e as palavras de decisão (abaixo)
`sublinear_tf`	True	Uma palavra repetida 10 vezes não pesa 10 vezes mais: o peso cresce mais devagar

Etapa 2: `LogisticRegression`. Dá um peso a cada termo e soma. Termos de peso positivo aumentam a probabilidade, os de peso negativo diminuem. O resultado passa por uma curva que o deixa entre 0 e 1.

Parâmetro	Valor	O que faz
`class_weight`	"balanced"	Como só 10% a 15% dos textos têm cada motivo, dá mais peso aos casos raros para o modelo não aprender a dizer sempre "não"
`C`	4	Controla o quanto o modelo pode se ajustar aos dados. Maior = mais ajuste. 4 é um valor moderado
`max_iter`	2000	Número máximo de passos de cálculo, para garantir que ele termine

O ajuste mais importante: `PALAVRAS_DE_DECISAO`

Termos como "decided", "churn", "cancel", "switch", "leaving", "ultimately" e "unfortunately" são **removidos do vocabulário**.

Sem esse ajuste, o modelo aprenderia um atalho: "texto de quem saiu tem queixa". Aí daria probabilidade baixa para qualquer satisfeito, que é justamente o público-alvo. Sem as palavras de decisão, ele é obrigado a reconhecer a **queixa em si** ("charges too high", "check inconvenient").

6. Notebook 2, fases 3 a 5: validação, o que o modelo aprendeu e escoragem

Nos clientes que ficaram, os modelos de preço e pagamento acertam bem (AUC 0,98). O de técnico deixa passar metade dos casos, e o de concorrente não tem o que medir.

Fase 3: validação cruzada

O problema. Se o modelo for avaliado nos mesmos textos em que treinou, ele parece melhor do que é, porque já viu as respostas.

A solução: previsões "fora da dobra" (out-of-fold).

1. `StratifiedKFold(5)` divide os 5.634 textos em 5 partes, mantendo a mesma proporção de casos com o motivo em cada parte.
2. O modelo treina em 4 partes e prevê a 5ª. Repete 5 vezes, trocando a parte separada.
3. `cross_val_predict` junta as 5 previsões. No fim, cada texto tem uma probabilidade dada por um modelo que **não** o viu no treino.

O papel do Churn aqui. Ele só serve para separar o grupo "quem ficou" na hora de calcular as métricas (`ficou`). Não entra no treino.

As três métricas

- **AUC.** Escolha um texto com o motivo e outro sem, ao acaso. AUC é a chance de o modelo dar probabilidade maior ao que tem o motivo. 0,5 é chute, 1,0 é perfeito.
- **Precisão** (corte em 0,5). Dos textos que o modelo marcou, quantos tinham o motivo. Alta precisão = poucos alarmes falsos.
- **Cobertura** (corte em 0,5). Dos textos com o motivo, quantos o modelo marcou. Alta cobertura = poucos casos perdidos.

Motivo	AUC (só quem ficou)	Precisão	Cobertura	Casos entre quem ficou
preço	0,98	98%	93%	178
pagamento	0,98	96%	86%	77
técnico	0,93	94%	44%	66
concorrente	não dá para medir	—	—	0

Fase 4: o que cada modelo aprendeu

O código lê os pesos (`lr.coef_`) e mostra os 10 termos de peso mais alto. Serve para conferir se o modelo aprendeu algo que faz sentido.

Motivo	Termos que mais aumentam a probabilidade
preço	high, higher, pricing, cost, expensive, price, worth
concorrente	better deal, affordable option, better value, providers better
pagamento	inconvenient, check inconvenient, issues electronic check, payment options
técnico	outages, unreliable, slow, speeds, lack internet service

Fase 5: escoragem do inference

Os quatro modelos, agora treinados com todo o `history` , são aplicados ao texto do `inference` com `predict_proba` . A tabela `saida` tem, por cliente:

- `prob_preco` , `prob_concorrente` , `prob_pagamento` , `prob_tecnico` : as quatro probabilidades.

- `motivo_mais_provavel` e `prob_maxima`: o motivo de maior probabilidade e o valor dela (`idxmax` e `max`).
- `status_texto` e `trecho_evidencia`: vindos das regras do notebook 1.

A tabela `monitorar` filtra quem **não** declarou saída e ordena pela `prob_maxima`, do maior para o menor. É a lista de trabalho do operador.

7. Resultados e leitura para o operador

Dos 1.045 clientes do `inference` que não declararam saída, 74 têm algum motivo com probabilidade acima de 0,5. Preço é o sinal mais frequente.

Motivo	Clientes com prob > 0,5	Clientes com prob > 0,8	Ação sugerida
preço	47	46	Desconto de fidelidade, revisão de plano, aviso antes de reajuste
pagamento	24	20	Oferecer débito automático ou cartão no lugar do cheque
técnico	10	8	Abrir chamado técnico, oferecer visita, mapear a região
concorrente	0	0	—

Como ler uma probabilidade

`prob_preco = 0,85` significa: **o feedback deste cliente muito provavelmente contém uma queixa de preço**. Não significa "85% de chance de cancelar por preço". O modelo mede o que está escrito, não o que o cliente vai fazer.

O operador deve sempre ler o `trecho_evidencia` antes de agir. A probabilidade aponta, a frase do cliente confirma.

O padrão que mais chamou atenção

Os clientes com maior `prob_preco` têm o mesmo perfil: estão na empresa há 4 a 6 anos, têm fibra, pagam mais de $100 e escrevem algo como *"a mensalidade é um pouco alta, mas acho que a qualidade justifica"*.

Hoje estão satisfeitos, mas já são **sensíveis a preço**. Um reajuste ou uma oferta do concorrente pode virar cancelamento. É o caso em que o contato preventivo mais compensa.

Por que concorrente fica vazio

Nenhum cliente que ficou mencionou uma oferta melhor de outra empresa. Nesta base, o concorrente só aparece no texto **depois** da decisão de sair. O modelo existe e funciona para quem saiu, mas não tem o que detectar no público de monitoramento.

8. Limitações, perguntas prováveis e glossário

Limitações

- **O modelo imita as regras.** Os AUCs de 0,93 a 0,98 medem o quanto ele concorda com as regras, não com a verdade. Se a regra errar, o modelo erra junto.
- **Falta validação humana.** O passo que falta é alguém ler uns 30 clientes que ficaram, com `prob_maxima` acima de 0,5, e dizer se a queixa está mesmo no texto.
- **Técnico perde casos.** Cobertura de 44%: queixas técnicas leves ("occasional slowdowns") passam. Baixar o corte de 0,5 para 0,3 aumenta a cobertura, com mais alarmes falsos.
- **Os textos desta base são muito padronizados.** Parecem seguir um molde. Com comentários reais, mais curtos e com gírias, as listas de palavras precisariam ser revistas.
- **Não sabemos quando o feedback é escrito.** Se, na prática, ele só aparece depois do cancelamento, a camada perde o sentido preventivo.

Perguntas prováveis do time

"Se o texto vaza o churn, por que estamos usando?" Ele não entra no modelo que escora o churn. Aqui ele só descreve o que o cliente já disse, para o operador agir. Vazamento é problema quando a informação é usada para prever a si mesma.

"Por que não clusterização?" No teste, os clusters agruparam perfis de cliente (fibra, contrato curto), não motivos. E cada texto cai em um único cluster, enquanto um cliente costuma ter 2 ou 3 queixas.

"Por que um modelo, se as regras já marcam?" O modelo dá uma probabilidade (ordena por força da queixa) e reconhece variações de frase que as regras não previam.

"O Churn foi usado em algum momento?" Só para medir o desempenho no grupo de quem ficou. Nunca no treino.

"As quatro probabilidades somam 1?" Não. São quatro modelos independentes, e um texto pode ter as quatro queixas, ou nenhuma.

"Por que tirar 'cancel' e 'decided' do vocabulário?" Para o modelo não aprender o atalho "quem saiu tem queixa" e funcionar em quem ficou.

Glossário

Termo	Significado
Vazamento (leakage)	Quando uma feature já contém a resposta que o modelo deveria prever
Regex	Padrão de busca de texto, com regras como "ou" (`|`) e "palavra inteira" (`\b`)
Gabarito / rótulo	A resposta certa de cada exemplo, que o modelo usa para aprender
TF-IDF	Peso de uma palavra num texto: alto se ela aparece ali e é rara no conjunto
N-grama	Sequência de N palavras seguidas ("check inconvenient" é um 2-grama)
Regressão logística	Modelo que soma pesos dos termos e transforma o total em probabilidade
Pipeline	Encadeia etapas (TF-IDF, depois regressão) para rodar como uma só
Validação cruzada	Treinar e testar em partes diferentes da base, várias vezes
Out-of-fold	Previsão feita por um modelo que não viu aquele exemplo no treino
AUC	Chance de o modelo dar nota maior a um caso positivo do que a um negativo
Precisão	Dos marcados pelo modelo, quantos estavam certos
Cobertura (recall)	Dos casos reais, quantos o modelo encontrou